

Entregables de cada equipo

Cada equipo deberá entregar los siguientes ficheros:

1. Presentación:

- Preferiblemente en formato **.pptx**. Puede ser en catalán, castellano o inglés.

2. Documento de descripción de los problemas:

- Debe incluir el **nombre de todos los integrantes del grupo**. Cada integrante debe indicar el problema que ha implementado.
- Para cada problema implementado, se debe proporcionar una **descripción detallada** que permita **reproducir el experimento**.
- No se debe omitir **ningún detalle relevante** del procedimiento o implementación.
- No hay número mínimo/máximo de páginas para el documento.
- Se valorará la originalidad del problema desarrollado.

3. Código fuente:

- Todos los ficheros **.py** correspondientes a las implementaciones de cada problema. Cada fichero debe tener como nombre un nemotécnico del problema que resuelve seguido del estudiante que lo ha implementado.

Herramientas de Python para código limpio, visualización y análisis de datos

Integrantes: Emma, Lua y Laia

Objetivos:

- Entender cómo algunas herramientas avanzadas de Python hacen más eficiente el trabajo con código científico o de Machine Learning.
- Describir las funcionalidades de la librería Seaborn para realizar visualización estadística de los datos.
- Describir las funcionalidades de la librería Pandas para manipular y analizar los datos.

Guión:

- Funciones “avanzadas” que ayudan a escribir mejor código: *decoradores*, *type hints* y *Context Managers*.
 - ✓ Qué es un *decorador* y cómo se usa para medir el tiempo que tarda un programa.
 - ✓ Cómo usar *type hints* para que el código sea más claro y fácil de revisar.

- ✓ Qué es un *context manager* (`with`) y cómo ayuda a liberar memoria o archivos automáticamente.

- Descripción de la librería Seaborn para la visualización estadística de los datos.
- Descripción de la librería de Pandas para manipular y analizar datos.
- Implementar tres problemas con aplicación física y de dificultad media-alta que incorporen las tres herramientas estudiadas.

Basic Python packages

Integrantes: Pau, Victor y Jan

Objetivo:

- Conocer y utilizar paquetes básicos de Python para resolver problemas físicos y matemáticos complejos, aprovechando librerías especializadas como SciPy, SymPy, PIL, SciKits y SWIG.

Guión:

- PIL (Python Imaging Library): procesamiento y manipulación de imágenes.
- SciKits: extensiones especializadas de SciPy para áreas específicas (ej. scikit-learn, scikit-image).
- SciPy: herramientas científicas para integración y optimización numérica, álgebra lineal, etc.
- SWIG: interfaz para conectar código C/C++ con Python (breve explicación y aplicaciones).
- SymPy: cálculo simbólico, álgebra computacional y manipulación de expresiones matemáticas.
- Implementar tres problemas físicos de dificultad medio-alta que involucren resolución numérica con SciPy, resolución simbólica con SymPy o integración de procesamiento de datos o imágenes con PIL.

Numpy

Integrantes: Sebastian, Miguel y David

Objetivo:

- Aprender a trabajar con arreglos y tensores, que son la base de los cálculos en Machine Learning.

Guión:

- Operaciones básicas con Numpy: sumas, productos, transposición, *slicing*.
- Qué es *broadcasting* y por qué acelera los cálculos.
- Implementar tres problemas con aplicación física y de dificultad media-alta usando Numpy, y que contenga una red neuronal que aprenda a predecir una relación sencilla, como por ejemplo la energía de un sistema físico. En cada implementación explicar los pasos a seguir para implementar una red neuronal:
 - ✓ Inicialización de la red
 - ✓ Forward propagation
 - ✓ Backpropagation vectorizada
 - ✓ Entrenamiento y predicción
 - ✓ Visualización de pérdida y resultados

Machine Learning

Integrantes: Gerard, Pau, Casandra y Joan

Objetivo:

- Entender las funcionalidades de la librería `scikit-learn` para predecir y clasificar datos con Python.

Guión:

- Qué es Machine Learning y en qué se diferencia del Deep Learning.
- Preprocesamiento de los datos (escalado, normalización y encoding):
`sklearn.preprocessing`
- División de datos, validación cruzada, búsqueda de hiperparámetros:
`sklearn.model_selection`
- Modelos supervisados para clasificación:
 - ✓ K-Nearest Neighbors (k-NN): `sklearn.neighbors`
 - ✓ Support Vector Machine (SVM): `sklearn.svm`
 - ✓ Random Forest (RF): `sklearn.ensemble`
 - ✓ Decision Tree (DT): `sklearn.tree`
 - ✓ Logistic Regression: `sklearn.linear_model`
- Modelos de regresión:
 - ✓ RF: En la librería `sklearn.ensemble` usa `RandomForestRegressor`
 - ✓ Extra Trees: En la librería `sklearn.ensemble` usa `ExtraTreesRegressor`
 - ✓ Gradient Boosting: En la librería `sklearn.ensemble` usa `GradientBoostingRegressor`, `HistGradientBoostingRegressor`
- Reducción de la dimensionalidad (PCA, NMF): `sklearn.decomposition`
- Cómo medir si un modelo es bueno (accuracy, F1-score, recall, precision, MCC (Mathews Correlation Coefficient)).

- Implementar tres problemas de dificultad medio-alta para predecir un valor físico usando `scikit-learn` y alguno de los modelos estudiados.

AI libraries en Python

Integrantes: Aina, Mariona, Carlota, Xavi, Ariadna y Paula

Objetivo:

- Comprender y comparar el uso de TensorFlow y PyTorch para construir, entrenar y evaluar modelos de Machine Learning y Deep Learning.

Guión:

- Diferencias principales entre TensorFlow y PyTorch.
- Manipulación de tensores en TensorFlow y PyTorch: Creación de tensores, operaciones básicas (suma, multiplicación, slicing).
 - ✓ Definición de modelos simples en TensorFlow (usando Keras API) y PyTorch.
 - ✓ Inicialización de pesos y capas.
 - ✓ Forward propagation en ambos frameworks.
- Backpropagation y optimización:
- Uso de autograd para cálculo automático de gradientes en TensorFlow y PyTorch.
- Definición de funciones de pérdida y optimizadores comunes (SGD, Adam).
- Ejecución del proceso de entrenamiento (loop de entrenamiento): forward, cálculo de pérdida, backpropagation, actualización de pesos.
- Implementación de 3 redes neuronales como por ejemplo, una red LSTM que utilice TensorFlow o Keras aplicada a un problema físico (ejemplo: Clasificación binaria usando Keras). Cada problema implementado será desarrollado por 2 miembros del equipo. Los 6 miembros deben participar en 1 problema y defenderlo.

Aprendizaje no supervisado y detección de anomalías

Integrantes: Matías, Martí, Uriel y Arnau

Objetivo:

- Entender cómo encontrar estructura y patrones en datos sin etiquetar mediante algoritmos de clustering y detección de anomalías.

Guión:

- Algoritmos de clustering: K-Means (`sklearn.cluster.KMeans`), DBSCAN (`sklearn.cluster.DBSCAN`) y clustering jerárquico.
- Modelos de mezcla gaussiana: `sklearn.mixture.GaussianMixture`.

- Detección de anomalías: Isolation Forest (`sklearn.ensemble.IsolationForest`).
- Cómo evaluar un clustering sin etiquetas: silhouette score y método del codo.
- Implementar cuatro problemas con aplicación física. Algunas sugerencias:
 - Fácil: agrupar estrellas por temperatura y luminosidad con K-Means y comparar los grupos con el diagrama de Hertzsprung-Russell.
 - Media: identificar cúmulos estelares en datos reales del satélite Gaia mediante DBSCAN (librería `astroquery`).
 - Difícil: detectar tránsitos de exoplanetas como anomalías en curvas de luz del telescopio Kepler (librería `lightkurve`).

Descubrimiento de leyes físicas con IA: regresión simbólica

Integrantes:

Objetivo:

- Aprender a utilizar funciones para descubrir la ecuación que explica unos datos, obteniendo modelos interpretables en lugar de modelos de caja negra.

Guión:

- Qué es la regresión simbólica y cómo busca expresiones matemáticas que ajusten los datos.
- Método SINDy (Sparse Identification of Nonlinear Dynamics) con la librería `pysindy`.
- Construcción de bibliotecas de funciones candidatas y regularización para obtener ecuaciones sencillas.
- Efecto del ruido en los datos y técnicas para mitigarlo.
- Implementar tres problemas con aplicación física, uno de cada nivel de dificultad. Algunas sugerencias:
 - **Fácil:** recuperar la tercera ley de Kepler ($T^2 \propto a^3$) a partir de datos orbitales de los planetas del sistema solar.
 - **Media:** descubrir las ecuaciones de un oscilador amortiguado o del sistema de Lotka-Volterra a partir de datos simulados.
 - **Difícil:** recuperar las ecuaciones de Lorenz a partir de datos con ruido y estudiar cómo afecta el nivel de ruido al resultado.